

MODULE 1 · LAB 1

DIAGNOSE & MAP A PROCESS

A build session — map the real process, find the chain, build the sandbox.

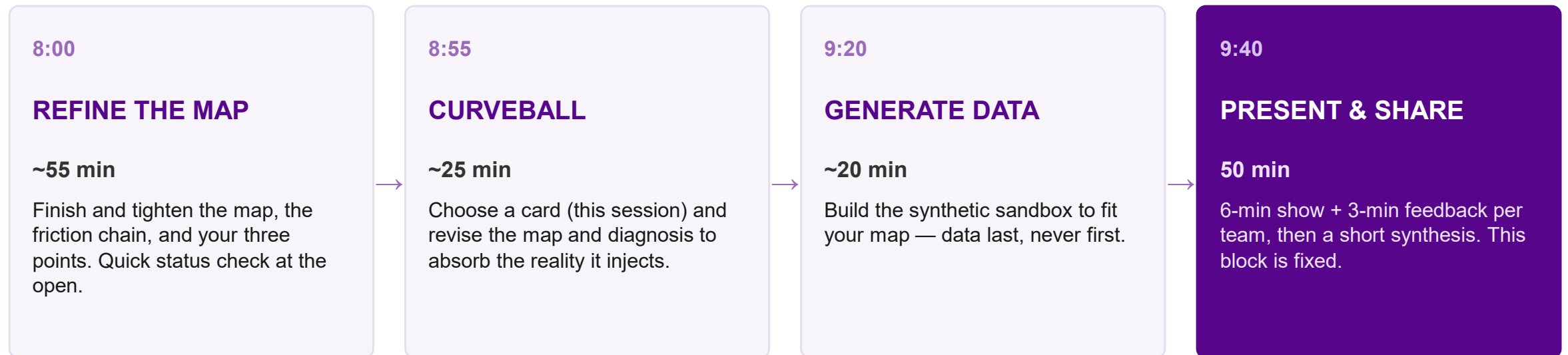
IE-GY 9113B · Systems Integration: From ERP to Agentic AI

Friday, September 18, 2026 · 8:00–10:30 AM

01

Mostly your time

About 10 minutes of me at the top. The rest of the two-and-a-half hours is your build and your presentations.



Front blocks flex; the 50-minute share-out at the end is fixed.

THE GATE. Your ½–1 page research brief was due before class, and it's checked. Without it you have no hypotheses to test — and the pattern library becomes a menu you copy instead of a mirror you check against. Bring it, or the lab doesn't work for your team.

What you're doing today

Four moves. The detail — BPMN shapes, the pattern library, the curveball deck, the Gemini prompts — lives in your study note (§4–§6). This slide is the map, not the territory.

① MAP THE REAL PROCESS

In BPMN. Mark ★ control points (where data is born), name every ♦ decision point (where work is routed), and trace the rejection / rework path — usually where the cost hides. Map what actually happens, not the ideal.

② MARK THREE DIAGNOSTIC POINTS

The worst friction (tag by Four Lens); a stable-rules decision point → your Lab 2 automation; a judgment decision point → your Lab 3 agent. Two of the three sit on decision points.

③ CHOOSE A CURVEBALL

A reality nobody told you about — this session you pick one from the deck (backup slide). Revise the map and diagnosis to absorb it: often a new rejection path, or a decision point that now behaves differently.

④ GENERATE DATA LAST

A synthetic sandbox fitted to the process you mapped. Verify every planted flaw survived — an LLM will often “helpfully” clean them away; confirm before you trust it.

FIND A CHAIN, NOT A PAIN. ≥ 3 causally-linked frictions, across ≥ 2 threads, ending in a board-level consequence. That is what a board funds.

The research brief (pre-work)

MIRROR, NOT TEMPLATE. Your company and chain will look different — the point is the shape, not the specifics. All figures synthetic.

REALITY BRIEF

Company / archetype

Citi Bike (Lyft), NYC bike-share — asset-heavy, high-frequency operations; ~mid-market operations archetype.

Integration stack

Live dock-status backbone + a dispatch/fleet system + an older nightly status feed still in the loop. Imperfectly integrated.

Business model

Memberships + per-ride fees; the model depends on a bike being available where and when a rider wants one.

Suspected pain / thread

Operations & Supply Chain (primary), with Customer (empty docks) and Cash (fleet cost) overlays.

FOUR LENSES ON ONE PROCESS (REBALANCING)

Frustration

A rider reaches a station and finds no bike, or nowhere to dock — the visible failure.

Time

Rebalancing is reactive — a truck goes only after a station empties, so it arrives late.

Cost

The fleet runs all day chasing shortfalls it could have anticipated.

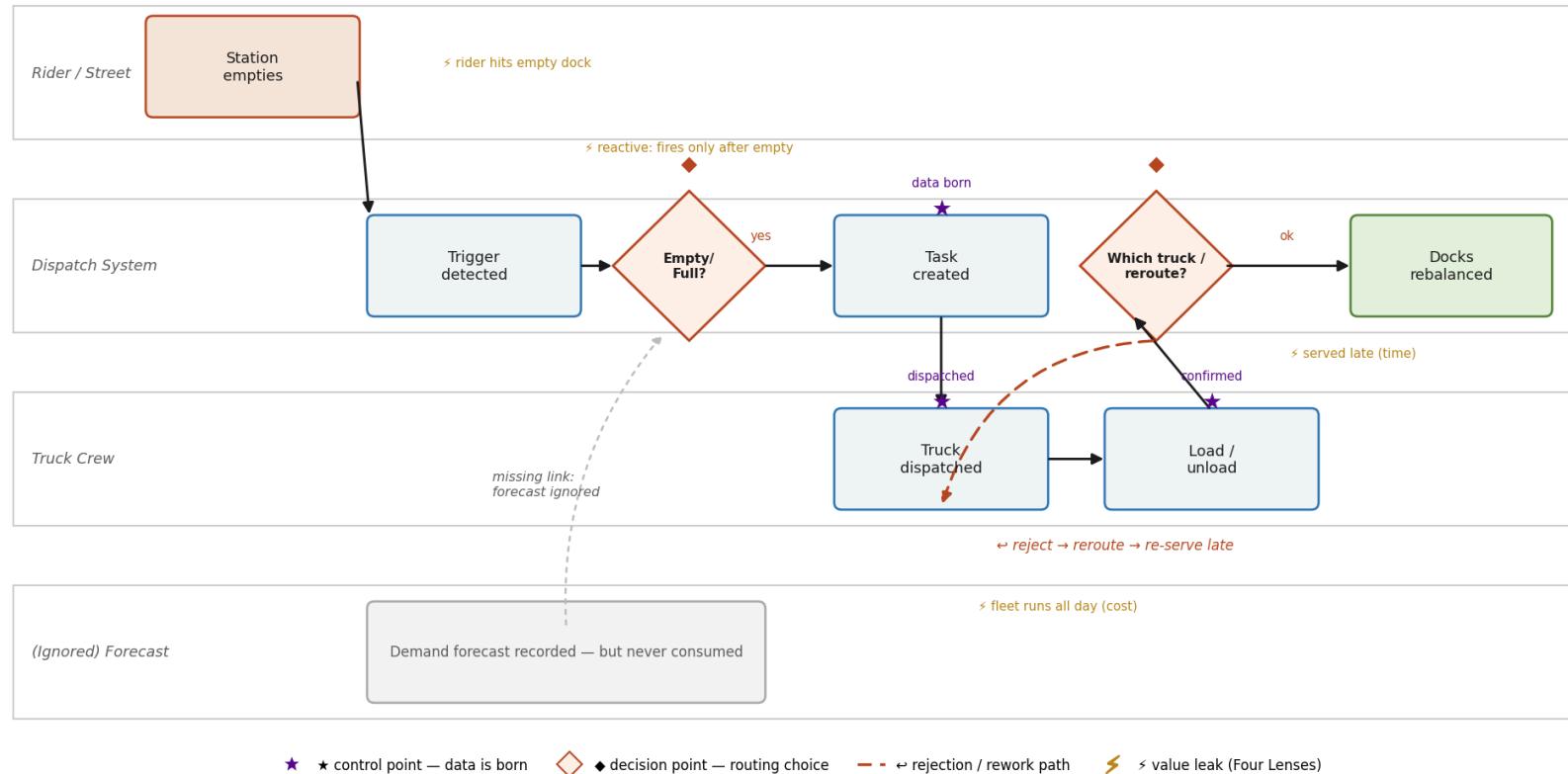
Quality

Some dispatches fire on stale (nightly) status, so trips are wasted or wrong.

FIRST-GUESS CHAIN (BROUGHT TO CLASS). Forecast ignored → reactive trigger fires only after a station empties → rider hits an empty dock → truck dispatched late → trips fail on arrival → rerouted, re-served late → fleet runs all day. Operations → Customer → Cash — a chain, not a single pain.

The annotated process map

Citi Bike rebalancing — Order-to-Cash-equivalent flow (reactive) · IE-GY 9113B Lab 1 worked example



Synthetic worked example — a mirror, not a template.

THE MISSING LINK. The bottom lane already records a demand forecast — but the reactive process never consumes it. That ignored control point is the root of the whole chain.

THE THREE POINTS ON THIS MAP

- ① **Worst friction** the rework loop — a failed dispatch rerouted and re-served late (Time + Cost).
- ② **Stable-rules** ♦ the Empty/Full trigger — clear logic. → Lab 2 (Make).
- ③ **Judgment** ♦ which truck / reroute when two stations compete. → Lab 3 (Relevance AI).

The one-page diagnosis

THE FRICTION CHAIN. Forecast recorded but never consumed → reactive trigger fires only after a station empties → rider hits an empty dock (Customer) → truck dispatched 10–40 min late (Time) → some dispatches fail on arrival: station full, status stale, or bikes already redistributed (Quality) → failed trips rerouted and re-served much later (rework) → fleet runs all day chasing shortfalls (Cost).

BOARD-LEVEL CONSEQUENCE

≈1 in 7 of 57 rebalancing tasks entered the rework path, each re-served ~40–70 min late. A structurally reactive operation spends fleet hours it needn't — and still fails riders.

② **STABLE-RULES → LAB 2 (Make).** Empty/Full trigger — automate it, then break it on the stale legacy feed.

③ **JUDGMENT → LAB 3 (Relevance AI).** Which truck / reroute when two stations compete — the call rules can't make.

THE PRIZE. If every link were addressed — consume the forecast the system already produces and pre-position bikes before stations empty — the same fleet closes more shortfalls at lower cost with fewer failures. That is the transformation the capstone will argue. Lab 1's job was only to prove the problem is real and locate exactly where it lives.

Illustrative figures from the team's synthetic sandbox — not real Citi Bike data.

The share-out: show four, answer three

Each team: a 6-minute show-and-tell, then 3 minutes of live feedback (~9 min per team, four teams). Don't narrate your build — show the destination.

SHOW FOUR THINGS

The annotated map

★ control, ◆ decision, and the rejection / rework path.

The friction chain

and its board-level cost — one number.

What the curveball changed

the reality that reshaped your diagnosis.

Your two targets

the Lab 2 automation and the Lab 3 agent.

ANSWER THREE QUESTIONS

- ① Where is the stable-rules decision point — your Lab 2 target?
- ② Which decision point needs judgment — your Lab 3 agent?
- ③ If it all worked, what bigger thing does it unlock?

The strongest chains — and the common trap

Name the two or three strongest chains in the room, and the one trap that showed up across teams.

TWO ANTI-PATTERNS TO CATCH

A PAIN, NOT A CHAIN

One symptom instead of ≥ 3 causally-linked frictions across ≥ 2 threads. “Invoices go out late” is a symptom; a chain shows how one friction causes the next, ending in a consequence a board would fund.

MAPPED THE SOLUTION, NOT REALITY

A suspiciously clean map means you drew what you wish existed. The rejection / rework path is where the cost hides — if your map doesn't have one, you mapped the ideal, not the real.

ETHICS THREAD. Look at the step you labeled as needing AI judgment. What is the human doing there that makes it hard to automate — and should it be automated at all? What is lost if it is? Hold that question; it returns when you build the Lab 3 agent.

And the data lesson: an LLM is a fast first draft you verify — the reflex you practise on synthetic data here is what keeps a real automation honest in Labs 2–3.

What you present today is slide S5

Drop today's annotated map + one-line consequence straight into **slide S5 (Backbone diagnosis)** of the board presentation template — you've just built a slide of your final board deck and the evidence for **Board Memo §3**. It's Milestone 1 (Diagnose).

Every lab feeds the deck this way — the board presentation and memo assemble as you go. Template: IE-GY_9113B_Board_Presentation_Template.

NEXT WEEK — RPA. Bring one thing: which decision point on your map is the stable-rules step you'd automate first — your Lab 2 target (→ S6, Memo §4).

SUBMIT BY MONDAY (GOOGLE WORKSPACE FOLDER)

Sandbox

Airtable / Sheets

Annotated map

★ ♦ + rejection path + 3 points + chain

One-page diagnosis

chain · consequence · prize

Chain-skeleton library

Causal shapes, not answers — a mirror to test your own chain against, never a menu to copy. Full detail: study note §6. Threads: C Customer · O Operations & Supply Chain · \$ Cash.

1 · Inventory-to-cash [O→C→\$]

Inaccurate stock → overselling → order rejected at fulfilment → manual rework → delayed invoicing → slow cash.

⇒ Working capital trapped; DSO inflated.

2 · Procurement blind spot [O→\$]

No spend visibility → maverick/duplicate buying → PO rejected/held → late reorder → production delay → expedited freight.

⇒ Margin erosion; uncontrolled spend.

3 · Multi-system reconciliation [O→C→\$]

Two systems disagree → records fail validation → staff reconcile by hand → errors propagate → wrong numbers out.

⇒ Audit risk; eroded trust in data.

4 · Returns & credit leakage [C→\$]

Manual returns → credit note rejected/held → disputed balances → customers withhold payment → cash & relationship damage.

⇒ Revenue leakage; churn risk.

5 · Quote-to-onboard drag [C→\$]

Quote/contract routed for manual approval → rejected/reworked → delayed onboarding → delayed first invoice → revenue late.

⇒ Slow time-to-revenue; forecast risk.

6 · Service-to-churn [C→\$]

Fragmented customer data → tickets misrouted/reopened → unresolved issues → dissatisfaction → churn → lost recurring revenue.

⇒ Retention loss; LTV decline.

Curveball options — choose one

This session, each team picks a curveball that genuinely challenges its process (instructor may veto a true misfit). Cards tagged DATA add a non-reconciling source to your sandbox. Full detail: study note §7.

1 · The hidden legacy system

DATA

A spreadsheet or old tool nobody mentioned still runs part of this — and its numbers disagree with the main system.

2 · The acquisition's parallel data

DATA

A recently acquired unit kept its own order / customer system; the two have never been merged.

3 · Nightly batch, not real time

DATA

A key data source only updates overnight, so decisions are made on yesterday's numbers.

4 · The undocumented reconciliation

DATA

One person fixes a recurring mismatch by hand every week — and they're about to leave.

5 · The vendor that only takes email/FTP

A critical partner can't do APIs; data moves by emailed file or FTP drop, with lag and errors.

6 · The compliance hold

A new audit / regulatory rule means a decision point now needs sign-off it didn't before — a new rejection route.

7 · The seasonal spike

Volume 5×'s for one period a year and the baseline process collapses under load.

8 · The duplicate-customer mess

DATA

The same customers exist under multiple IDs across systems, so totals never quite add up.

9 · The offline edge case

Part of the process happens offline (paper, phone, a remote site) and only gets keyed in later.

10 · The well-meaning workaround

A team built a shadow process to cope with the system's limits; it “works” but hides the real problem.